

FLASHIN DELIVERY
LAST-MILE ROUTE PLANNING SYSTEM

Project report submitted in fulfilment of the requirements for the Degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

By

SHAIK NAZEER

Registration Number: _____

Subject

INT219 - FRONTEND WEB DEVELOPMENT

School of Computer Science and Engineering

Lovely Professional University

Phagwara, Punjab (India)

May 2026

DECLARATION

I hereby declare that the project work entitled "**FlashIn Delivery - Last-Mile Route Planning System**" is an authentic record of my own work carried out as a requirement of the project submission for the award of B. Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara. The information furnished in this project report is based on my own study, implementation, testing, and project development work.

This project has been developed as a web-based delivery management system that supports delivery center management, driver management, customer order handling, route creation, and operational tracking for last-mile delivery.

Name of Student: Shaik Nazeer

Registration Number: _____

Date: 16 May 2026

Signature of Student

Signature of Faculty

CERTIFICATE

This is to certify that the declaration statement made by the student is correct to the best of my knowledge and belief. The student has completed the project titled "**FlashIn Delivery - Last-Mile Route Planning System**" under guidance and supervision. The present work is the result of the student's original investigation, effort, design, and study.

No part of this work has been submitted for any other degree at any university. The project is fit for submission and partial fulfilment of the conditions for the award of B. Tech degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

Signature and Name of the Staff: _____

Designation: _____

School of Computer Science and Engineering,
Lovely Professional University,
Phagwara, Punjab.

Date: _____

TABLE OF CONTENTS

1. Introduction	6
1.1 Overview of the Website	6
1.2 Need and Importance of Last-Mile Delivery	6
1.3 Objectives of the Website	6
2. Project Modules and Features	7
2.1 Dashboard Module	7
2.2 Delivery Centers, Drivers, Orders, and Routes	7
2.3 Key Features	7
3. Literature Survey	8
3.1 Last-Mile Delivery Management	8
3.2 Route Planning and Operational Tracking	8
3.3 Web-Based CRUD Systems	8
4. Flow of the Work	9
4.1 User Journey on the Website	9
4.2 Navigation and Workflow	9
5. Comparative Analysis	10
6. Software and Tools Utilized	11
7. System Analysis and Requirements	13
8. Implementation Details	14
9. Testing and Validation	15
10. Advantages, Limitations, and Applications	16
11. Project Screenshots	17
12. Code Screenshots	19
13. Conclusions and Future Directions	22
14. References and GitHub Link	23

ABSTRACT

FlashIn Delivery is a web-based last-mile route planning and delivery management system. The main purpose of the project is to help a local delivery organization manage the movement of customer orders from stores or warehouses to final customer addresses. The system provides a centralized interface for managing delivery centers, delivery partners, customer orders, and delivery routes.

The project was developed using Laravel, Blade, Bootstrap, PHP, and SQLite. It follows a simple Model-View-Controller structure. The dashboard provides operational metrics such as total orders, delivered orders, active drivers, route counts, and recent delivery activity. CRUD functionality is provided for all major entities so that the user can create, view, update, and delete operational records.

The application is useful for college-level demonstration of full-stack web development, database design, route management, form validation, and responsive user interface design. It also demonstrates how delivery operations can be digitized to reduce manual work and improve visibility.

Keywords: Last-mile delivery, route planning, Laravel, CRUD, delivery management, dashboard, logistics.

1. INTRODUCTION

1.1 OVERVIEW OF THE WEBSITE

FlashIn Delivery is designed as an administrative web application for managing local delivery operations. The website allows the operator to maintain records of fulfillment centers, drivers, orders, and routes. It is especially suitable for grocery, essentials, and local retail delivery businesses where orders must be assigned and tracked quickly.

1.2 NEED AND IMPORTANCE OF LAST-MILE DELIVERY

Last-mile delivery is the final stage in the delivery process, where goods are transported from a hub or store to the customer. This stage is important because it directly affects customer satisfaction, delivery time, and operational cost. Manual tracking can lead to delays, duplicate work, poor assignment decisions, and limited visibility. A digital system helps the organization keep data organized and improves decision-making.

1.3 OBJECTIVES OF THE WEBSITE

- To provide a simple dashboard for monitoring delivery operations.
- To manage delivery centers, warehouses, and local stores.
- To create and maintain driver details with availability status.
- To create customer orders and assign them to drivers and routes.
- To create route records with start location, end location, distance, and estimated time.
- To provide a professional and responsive interface for project demonstration.

2. PROJECT MODULES AND FEATURES

2.1 DASHBOARD MODULE

The dashboard is the first screen of the website. It displays important summary values including total orders, delivered orders, active drivers, total routes, pending orders, assigned orders, delivery centers, recent orders, and recent routes. This gives the user a quick operational view.

2.2 MAIN MODULES

Module	Description
Delivery Centers	Stores, warehouses, and grocery hubs can be added and managed with location, contact information, type, status, and coordinates.
Drivers	Delivery partners can be stored with phone number, vehicle type, license number, assigned center, and working status.
Orders	Customer orders include order number, customer details, delivery address, items, status, assigned driver, store, and route.
Routes	Routes store route name, driver, start and end locations, coordinates, estimated distance, estimated time, and current status.

2.3 KEY FEATURES

- Responsive navigation with active page highlighting.
- Professional Bootstrap-based user interface.
- Form validation for required fields and database relationships.
- Pagination for large lists of orders, routes, drivers, and centers.
- Status badges for quick identification of delivery progress.
- SQLite database support for easy local setup.

3. LITERATURE SURVEY

3.1 LAST-MILE DELIVERY MANAGEMENT

Last-mile delivery systems focus on organizing orders, delivery partners, and dispatch routes. Existing logistics platforms show that centralized tracking reduces operational confusion and improves customer service. Delivery systems generally require a clear relationship between hubs, drivers, orders, and routes.

3.2 ROUTE PLANNING AND OPERATIONAL TRACKING

Route planning is important because delivery time and distance affect cost. Modern systems often combine route information, driver assignment, and order status. FlashIn implements the foundational version of this workflow by storing route distance, estimated time, start and end coordinates, and route status.

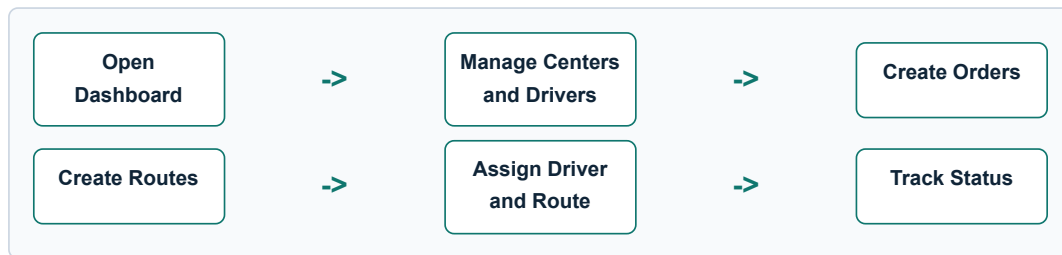
3.3 WEB-BASED CRUD SYSTEMS

CRUD applications are widely used in business software because they allow users to create, read, update, and delete records. Laravel is useful for such systems because it provides routing, controllers, validation, Blade templates, database migrations, and Eloquent ORM. These features helped in developing FlashIn as a structured and maintainable project.

4. FLOW OF THE WORK

4.1 USER JOURNEY ON THE WEBSITE

The user begins from the dashboard, where operational statistics are shown. From the navigation bar, the user can open Centers, Drivers, Orders, or Routes. The user first creates delivery centers, then adds drivers assigned to those centers. After that, customer orders can be created and associated with available centers, drivers, and routes.



4.2 NAVIGATION AND WORKFLOW

The website contains a clear top navigation bar with Dashboard, Centers, Drivers, Orders, and Routes. Every module contains listing, creation, editing, detail view, and deletion actions. The forms use Laravel validation to prevent invalid entries, and success messages are shown after important operations.

5. COMPARATIVE ANALYSIS OF DELIVERY APPLICATIONS

5.1 REVIEW OF EXISTING DELIVERY PLATFORMS

Popular delivery platforms such as Blinkit, Zepto, Swiggy Instamart, and Dunzo provide fast local delivery experiences. They include advanced features such as live tracking, automated dispatching, inventory management, payment systems, and customer mobile applications. FlashIn is a smaller academic system focused on the core administrative workflow.

5.2 FEATURE COMPARISON

Feature	Existing Apps	FlashIn Project
Order Management	Advanced customer-facing order lifecycle.	Admin can create, edit, assign, and update orders.
Driver Management	Driver apps with live status and GPS.	Admin can maintain driver records and status.
Route Planning	Automated routing and live tracking.	Manual route creation with distance and estimated time.
Dashboard	Advanced analytics and monitoring.	Simple operational metrics and recent activity.
Technology Scope	Large-scale cloud and mobile infrastructure.	Laravel web application suitable for academic demonstration.

5.3 STRENGTHS AND LIMITATIONS

The major strength of FlashIn is its simplicity. It is easy to understand, run, and demonstrate. Its limitation is that it does not yet include real-time GPS tracking, automated route optimization, customer login, or payment integration. These can be added in future versions.

6. SOFTWARE AND TOOLS UTILIZED

6.1 OVERVIEW OF SOFTWARE USED

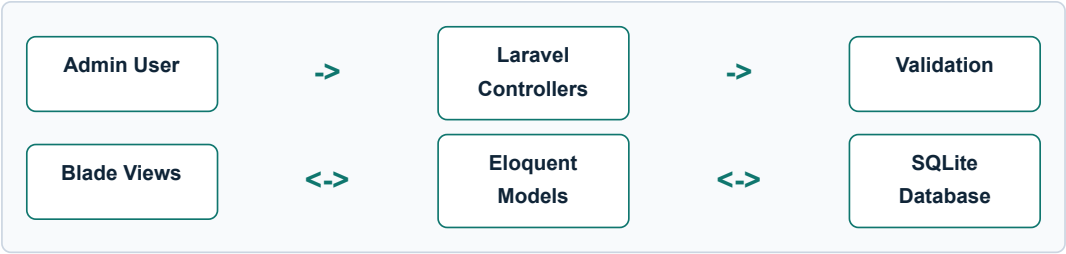
Software / Tool	Use in Project
Laravel	Backend framework used for routing, controllers, validation, models, migrations, and Blade views.
PHP	Server-side programming language for application logic.
Blade	Laravel templating engine used to build reusable web pages.
Bootstrap	Responsive layout and UI components.
Font Awesome	Icons used in navigation, dashboard, and action buttons.
SQLite	Local database used for storing project records.
Git and GitHub	Version control and remote repository hosting.
Composer	PHP dependency manager.
Vite / NPM	Frontend asset tooling available in the Laravel setup.

6.2 DATABASE DESIGN

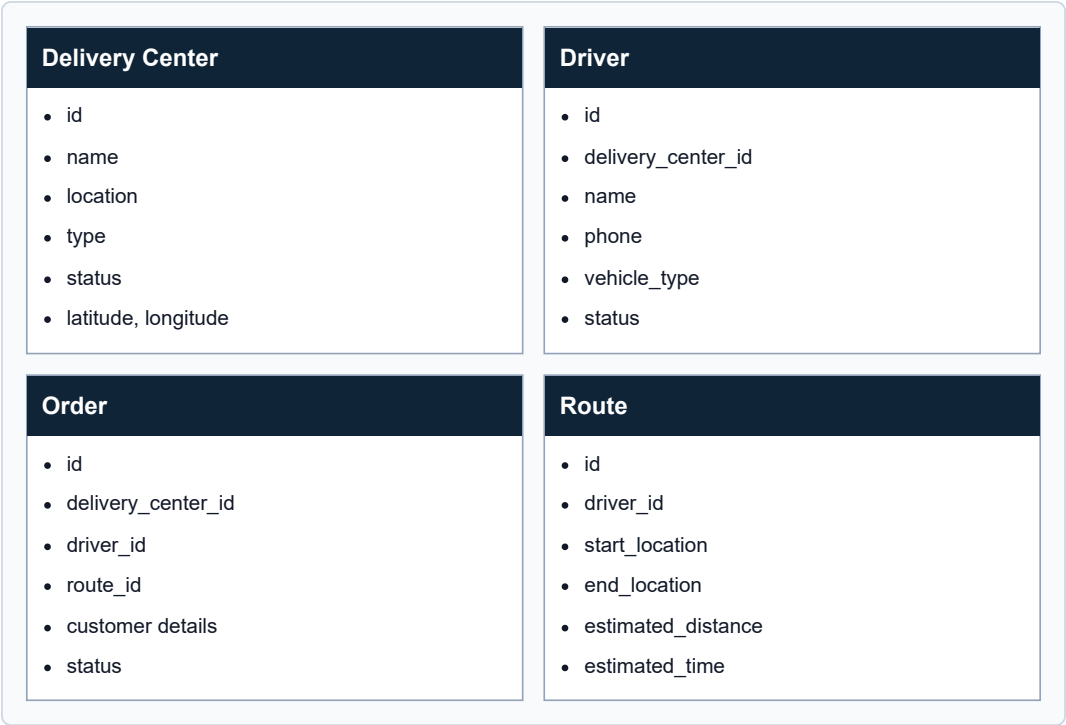
The database contains four main project tables: `delivery_centers`, `drivers`, `orders`, and `routes`. These tables are connected through foreign keys, which maintain relationships between stores, drivers, route assignments, and customer orders.

6.3 DIAGRAMS FOR WEBSITE DESIGN

DATA FLOW DIAGRAM



ENTITY RELATIONSHIP OVERVIEW



Relationship summary: One delivery center can have many drivers and many orders. One driver can be assigned to many routes and orders. One route can be connected to multiple orders through route assignment.

7. SYSTEM ANALYSIS AND REQUIREMENTS

7.1 PROBLEM STATEMENT

Many small delivery businesses still manage daily orders using notebooks, spreadsheets, phone calls, and messaging applications. This method becomes difficult when order volume increases because store managers need to know which driver is available, which order is pending, which route is active, and how many orders are already completed. Lack of a centralized system can create confusion, repeated communication, delayed delivery, and poor tracking of operational records.

FlashIn solves this problem by providing a single web interface for the main delivery workflow. The operator can create delivery centers, register drivers, create orders, plan routes, and monitor statuses from one place. The project focuses on the administrative side of last-mile delivery and gives a clear base for future real-time tracking and route optimization.

7.2 FUNCTIONAL REQUIREMENTS

- The system should allow the admin to add, update, view, and delete delivery centers.
- The system should store driver details such as name, phone, vehicle type, license number, assigned center, and status.
- The system should allow creation of customer orders with address, items, store, driver, route, and order status.
- The system should allow route creation with start location, end location, coordinates, distance, time, driver, and route status.
- The dashboard should show total orders, delivered orders, active drivers, total routes, centers, recent orders, and recent routes.
- The system should validate input forms and show error messages when required data is missing.

7.3 NON-FUNCTIONAL REQUIREMENTS

Requirement	Description
Usability	The interface should be simple enough for an operator to use without technical training.
Responsiveness	The website should work on desktop and smaller screens using responsive Bootstrap layout.
Maintainability	The code should follow Laravel MVC structure so that modules can be updated independently.
Reliability	Database relationships and validation should reduce invalid records.
Portability	The system should run locally using PHP, Composer, SQLite, and Laravel.

8. IMPLEMENTATION DETAILS

8.1 ARCHITECTURE USED

The project follows the Model-View-Controller architecture provided by Laravel. In this pattern, models represent database entities, views present information to the user, and controllers handle requests, validation, and communication between models and views. This separation makes the project easy to understand and improves maintainability.

Layer	Files / Components	Purpose
Model	DeliveryCenter, Driver, Order, Route	Defines data relationships and database interaction.
View	Blade templates inside resources/views	Displays dashboard, lists, forms, detail pages, and status badges.
Controller	DashboardController, DeliveryCenterController, DriverWebController, OrderWebController, RouteWebController	Handles page requests, form validation, CRUD operations, and redirects.
Database	Laravel migrations and SQLite	Stores project data in structured tables.

8.2 CONTROLLER LOGIC

Each module uses a controller that provides standard resource methods: index, create, store, show, edit, update, and destroy. For example, the order controller loads active delivery centers, available drivers, and route records before displaying the order creation form. When a form is submitted, Laravel validation checks the data before creating or updating the record in the database.

8.3 USER INTERFACE IMPLEMENTATION

The interface is built using Blade, Bootstrap, Font Awesome, and a custom theme stylesheet. The design uses a professional operations-console look with a navy header, white content cards, status badges, responsive tables, and clean forms. Common layout elements such as the navigation bar, footer, error messages, and success alerts are placed in the shared layout file so that all pages maintain the same appearance.

8.4 DATABASE RELATIONSHIPS

- A delivery center can have multiple drivers.
- A delivery center can have multiple orders.
- A driver can be assigned to multiple orders and routes.
- An order can optionally belong to a route and driver.
- A route can be assigned to a driver and can be connected to orders.

9. TESTING AND VALIDATION

9.1 TESTING APPROACH

Testing was performed by running the Laravel project locally and checking each major module. The dashboard, centers page, drivers page, orders page, and routes page were opened in the browser to confirm that the pages load correctly. Forms were checked by entering valid and invalid data to confirm that validation rules work as expected.

9.2 TEST CASES

Test Case	Expected Result	Status
Open dashboard URL	Dashboard should load and show delivery metrics.	Passed
Create delivery center	New center should be saved and displayed in the centers list.	Passed
Create driver	Driver should be saved with vehicle and assigned center details.	Passed
Create order	Order number and customer order details should be stored.	Passed
Create route	Route should be saved with driver, locations, distance, time, and status.	Passed
Submit empty required fields	Validation message should be shown and record should not be saved.	Passed
Delete a record	Selected record should be removed after confirmation.	Passed

9.3 VALIDATION RULES

Laravel validation is used to protect the application from incomplete or invalid input. Required fields include customer name, phone, delivery address, delivery center, route name, route distance, route time, and status. Foreign key validation ensures that selected drivers, centers, and routes already exist in the database.

9.4 RESULT OF TESTING

The system successfully supports the expected CRUD workflow. The pages load correctly, records are stored in SQLite, and the user receives success or validation messages based on the performed action. This confirms that the core last-mile delivery management workflow is functional.

10. ADVANTAGES, LIMITATIONS, AND APPLICATIONS

10.1 ADVANTAGES

- Centralizes delivery center, driver, route, and order information.
- Reduces manual record keeping and makes data easier to search and update.
- Provides a quick dashboard summary for operational monitoring.
- Uses a simple and professional interface suitable for academic demonstration.
- Uses Laravel MVC, which makes the project structured and scalable.
- Can be run locally with SQLite, so setup is simple for evaluation.

10.2 LIMITATIONS

- The current version does not include login authentication or user roles.
- Route optimization is manual and does not calculate the best path automatically.
- Live GPS tracking is not implemented in this version.
- The system does not include customer mobile application features.
- Payment, invoice, and notification modules are outside the current scope.

10.3 REAL-WORLD APPLICATIONS

FlashIn can be used as a basic administrative system for small courier businesses, grocery delivery stores, campus delivery teams, local retail logistics, and academic demonstrations. It is also useful as a foundation for developing larger logistics applications because its entities match real delivery operations: centers, drivers, orders, and routes.

10.4 LEARNING OUTCOMES

Through this project, important concepts of web development were practiced, including MVC architecture, database migrations, CRUD operations, form validation, responsive layout design, route definitions, template reuse, version control, and project deployment preparation.

11. PROJECT SCREENSHOTS

11.1 DASHBOARD SCREEN

The dashboard gives an overview of delivery activity. It shows total orders, delivered orders, active drivers, total routes, order status summaries, recent orders, and recent routes.

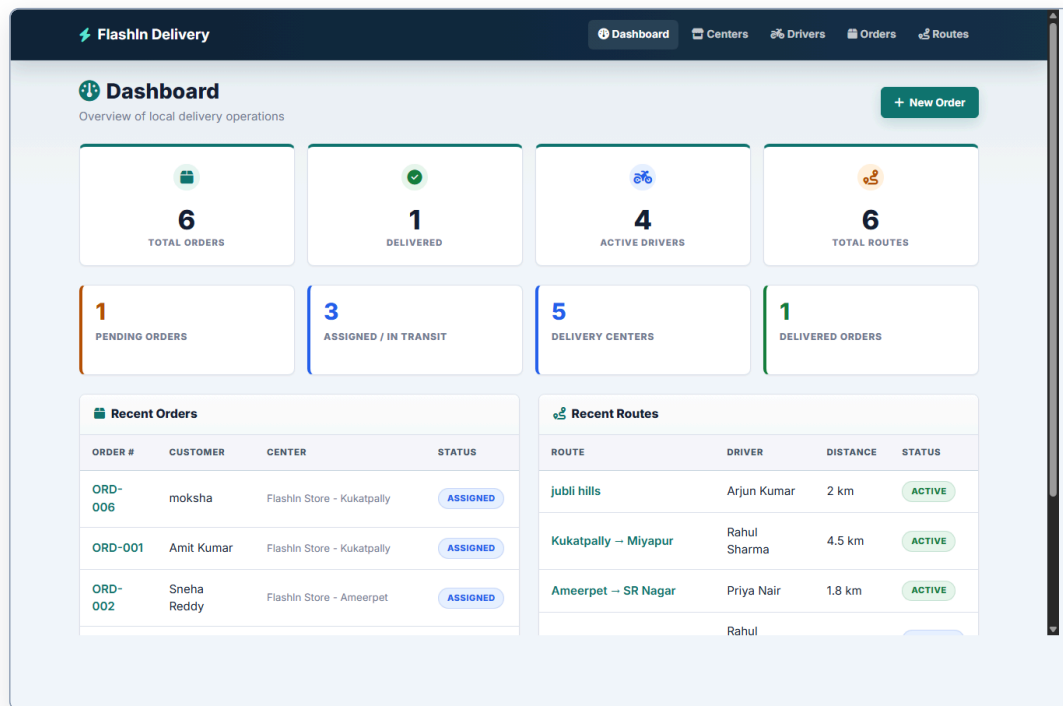


Figure 1: Dashboard overview of delivery operations.

11.2 ORDERS SCREEN

The orders page displays customer orders with customer name, item summary, delivery address, assigned store, delivery partner, status, and available actions such as edit and delete.

FlashIn Delivery

DashboardCentersDriversOrdersRoutes

Orders

Quick delivery orders for groceries, essentials, and local retail

+ New Order

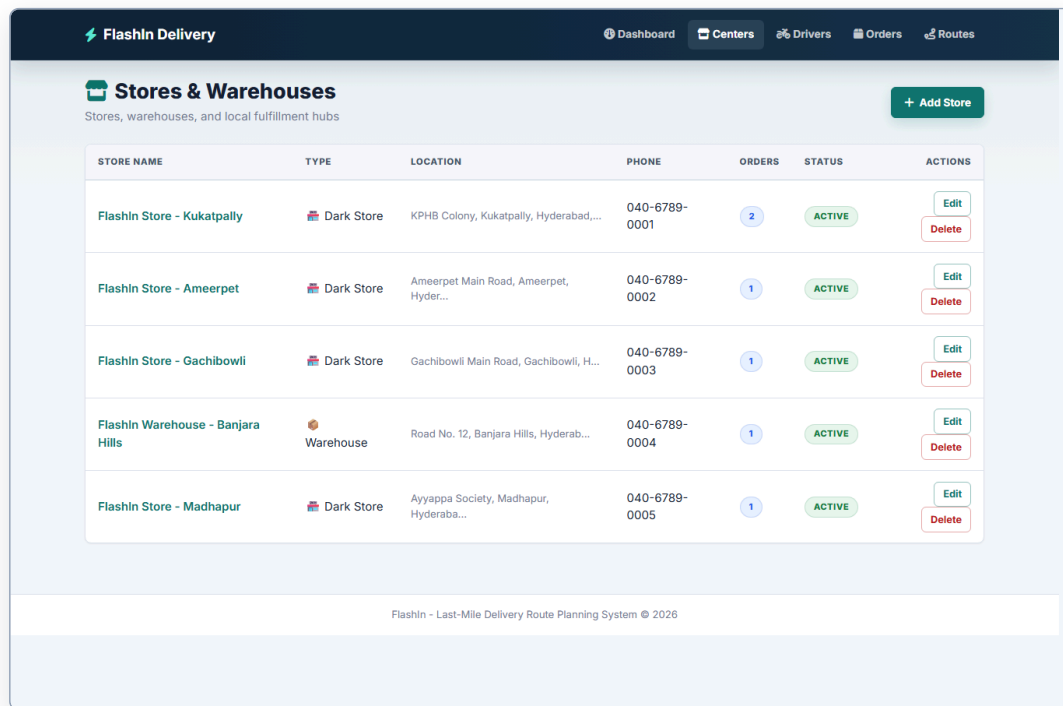
ORDER #	CUSTOMER	ITEMS	DELIVERY ADDRESS	STORE	PARTNER	STATUS	ACTIONS
ORD-006	moksha	milk	R.R.peta, eluru, AP, I...	FlashIn Store - Kukatpally	Arjun Kumar	ASSIGNED	Edit Delete
ORD-001	Amit Kumar	Milk, Bread, Eggs, B...	Flat 203, Green Valley...	FlashIn Store - Kukatpally	Rahul Sharma	ASSIGNED	Edit Delete
ORD-002	Sneha Reddy	Rice 5kg, Dal, Cooki...	House No. 45, SR Nagar...	FlashIn Store - Ameerpet	Priya Nair	ASSIGNED	Edit Delete
ORD-003	Rajesh Verma	Cold Drinks, Chips,...	Office 301, Tech Tower...	FlashIn Store - Gachibowli	Unassigned	PENDING	Edit Delete
ORD-004	Pooja Singh	Fruits, Vegetables,...	Flat 12, Jubilee Encla...	FlashIn Warehouse - Banjara Hills	Vikram Patel	PICKED UP	Edit Delete
ORD-005	Khushi Patel	Detergent, Soap, Sha...	Plot 88, Kondapur Main...	FlashIn Store - Madhapur	Arjun Kumar	DELIVERED	Edit Delete

Figure 2: Orders listing page with status and action controls.

11. PROJECT SCREENSHOTS CONTINUED

11.3 DELIVERY CENTERS SCREEN

The centers page helps the admin manage stores, warehouses, and fulfillment hubs. It includes center type, location, contact details, order count, and current status.



STORE NAME	TYPE	LOCATION	PHONE	ORDERS	STATUS	ACTIONS
FlashIn Store - Kukatpally	Dark Store	KPHB Colony, Kukatpally, Hyderabad,...	040-6789-0001	2	ACTIVE	Edit Delete
FlashIn Store - Ameerpet	Dark Store	Ameerpet Main Road, Ameerpet, Hyder...	040-6789-0002	1	ACTIVE	Edit Delete
FlashIn Store - Gachibowli	Dark Store	Gachibowli Main Road, Gachibowli, H...	040-6789-0003	1	ACTIVE	Edit Delete
FlashIn Warehouse - Banjara Hills	Warehouse	Road No. 12, Banjara Hills, Hyderab...	040-6789-0004	1	ACTIVE	Edit Delete
FlashIn Store - Madhapur	Dark Store	Ayyappa Society, Madhapur, Hyderaba...	040-6789-0005	1	ACTIVE	Edit Delete

FlashIn - Last-Mile Delivery Route Planning System © 2026

Figure 3: Delivery centers and fulfillment hubs.

11.4 DRIVERS AND ROUTES SCREENS

The drivers and routes modules are used to manage delivery partners and route planning records. Drivers can be assigned to centers, while routes store start location, end location, estimated distance, estimated time, and route status.

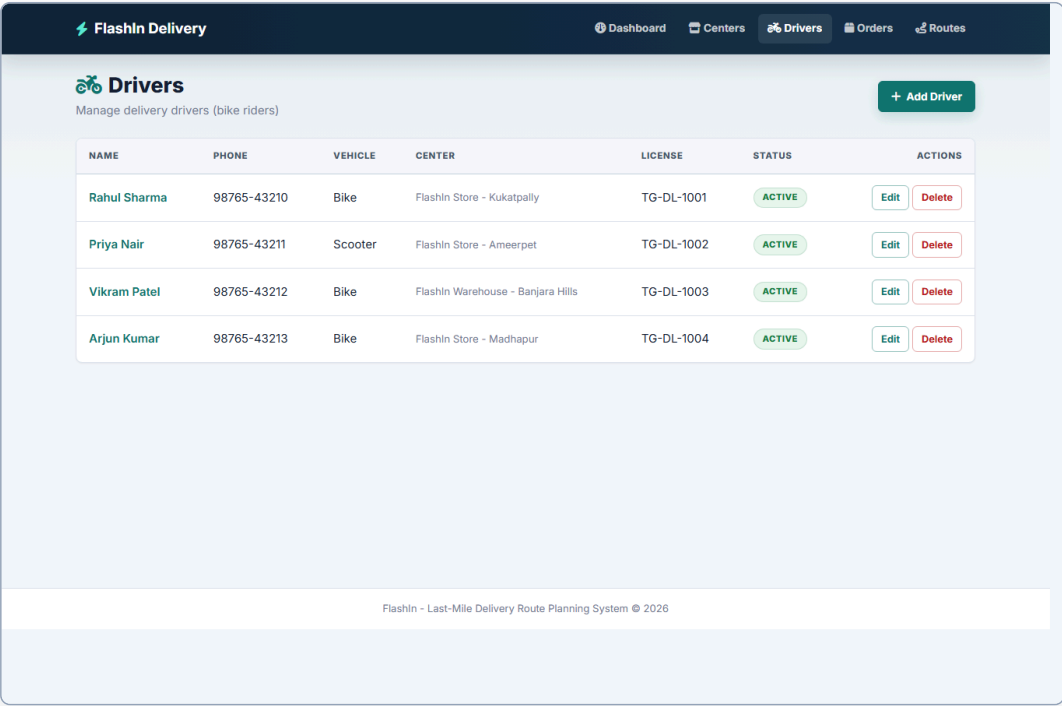


Figure 4: Driver management page.

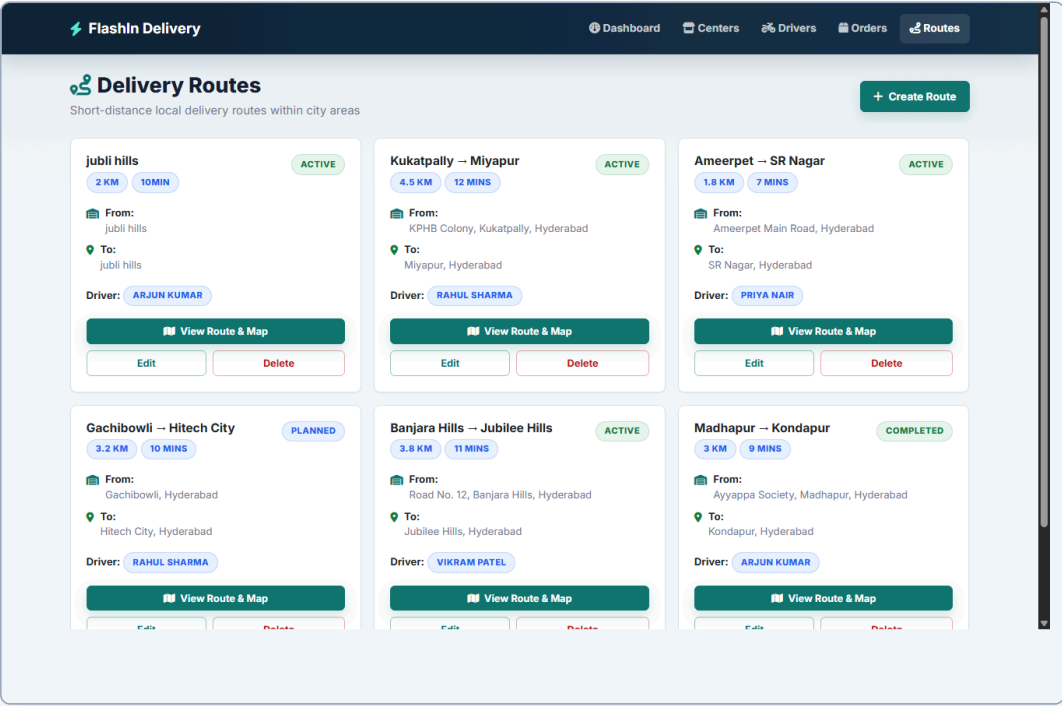
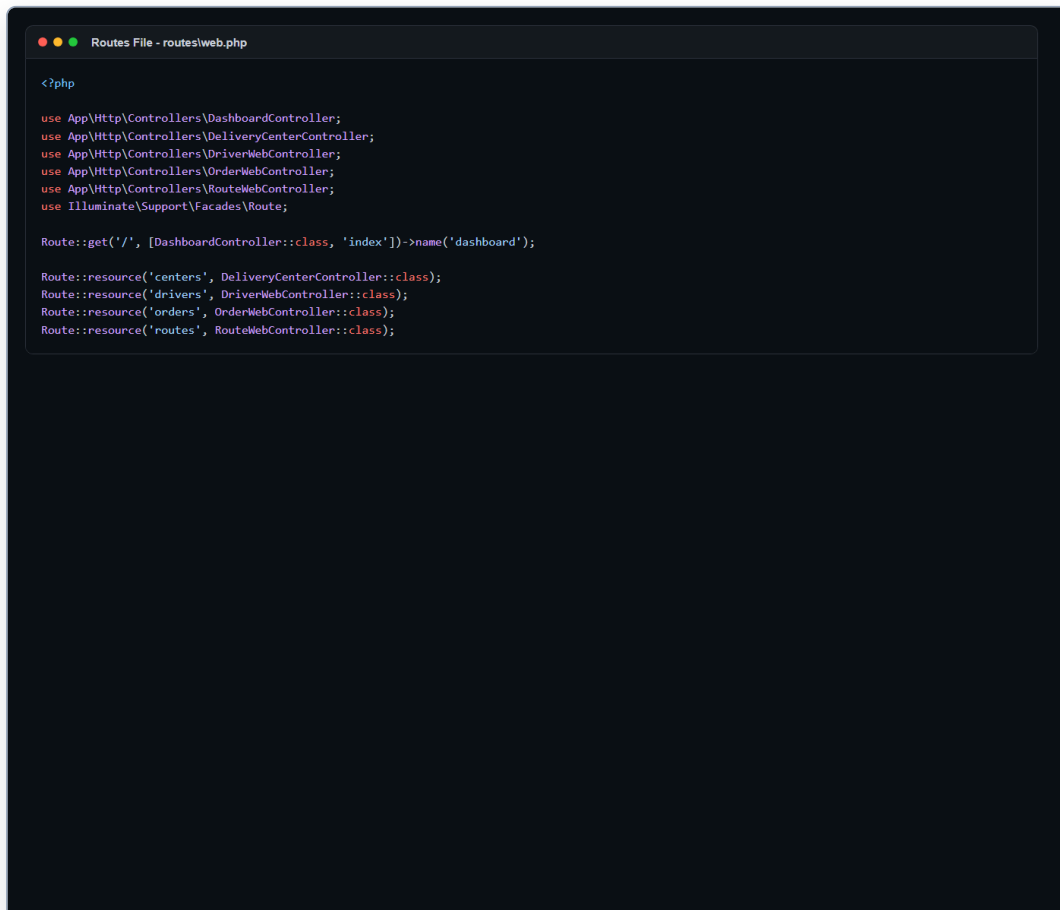


Figure 5: Route planning page.

12. CODE SCREENSHOTS

12.1 ROUTES FILE

The routes file defines the main web routes of the application. It maps the dashboard and CRUD resources to their respective Laravel controllers.



```
<?php

use App\Http\Controllers\DashboardController;
use App\Http\Controllers\DeliveryCenterController;
use App\Http\Controllers\DriverWebController;
use App\Http\Controllers\OrderWebController;
use App\Http\Controllers\RouteWebController;
use Illuminate\Support\Facades\Route;

Route::get('/', [DashboardController::class, 'index'])->name('dashboard');

Route::resource('centers', DeliveryCenterController::class);
Route::resource('drivers', DriverWebController::class);
Route::resource('orders', OrderWebController::class);
Route::resource('routes', RouteWebController::class);
```

Figure 6: Laravel route definitions for dashboard and modules.

12.2 DASHBOARD CONTROLLER

The dashboard controller collects counts and recent records from the database and sends them to the dashboard view.

```
Dashboard Controller - app\Http\Controllers\DashboardController.php

<?php

namespace App\Http\Controllers;

use App\Models\DeliveryCenter;
use App\Models\Driver;
use App\Models\Order;
use App\Models\Route;
use Illuminate\View\View;

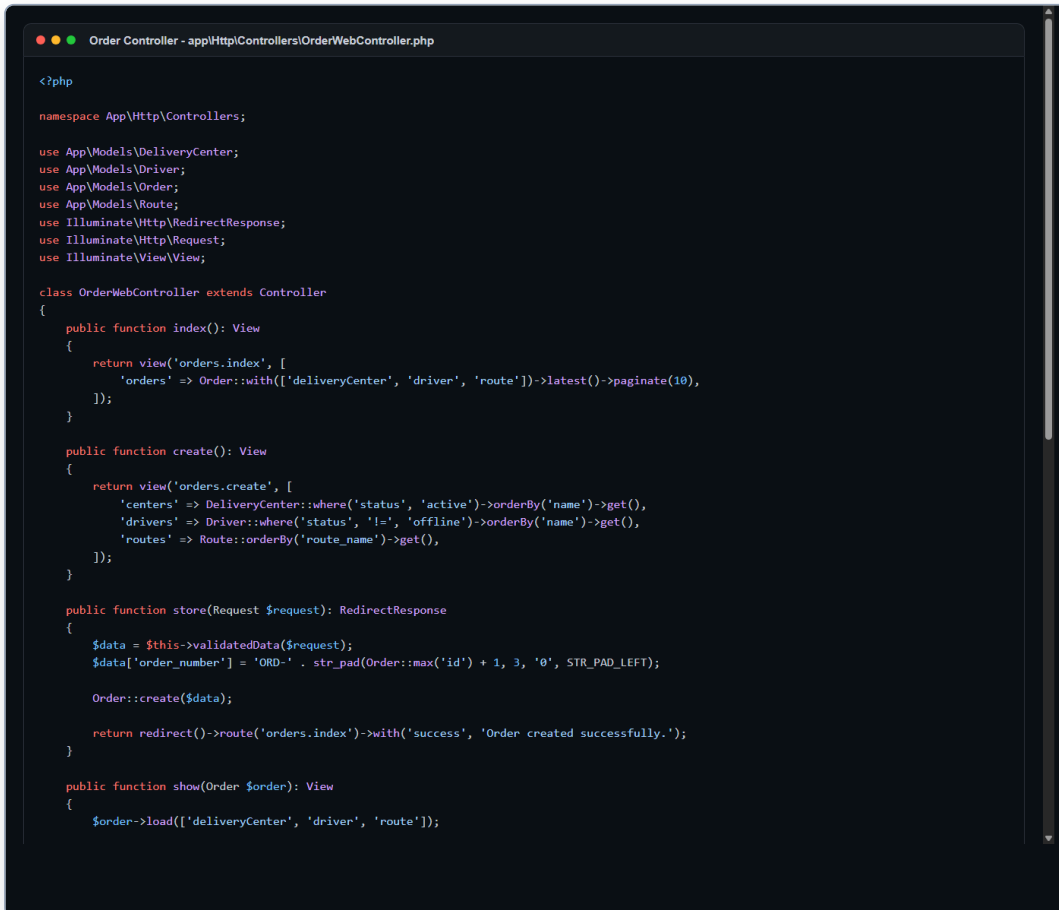
class DashboardController extends Controller
{
    public function index(): View
    {
        return view('dashboard', [
            'totalOrders' => Order::count(),
            'deliveredOrders' => Order::where('status', 'delivered')->count(),
            'pendingOrders' => Order::where('status', 'pending')->count(),
            'assignedOrders' => Order::where('status', 'assigned')->count(),
            'activeDrivers' => Driver::where('status', 'active')->count(),
            'totalRoutes' => Route::count(),
            'totalDeliveryCenters' => DeliveryCenter::count(),
            'recentOrders' => Order::with(['deliveryCenter', 'driver'])->latest()->limit(5)->get(),
            'recentRoutes' => Route::with('driver')->latest()->limit(5)->get(),
        ]);
    }
}
```

Figure 7: DashboardController code.

12. CODE SCREENSHOTS CONTINUED

12.3 ORDER CONTROLLER

The order controller handles order listing, creation, editing, validation, and deletion. It also loads related centers, drivers, and routes for order forms.



```
<?php

namespace App\Http\Controllers;

use App\Models\DeliveryCenter;
use App\Models\Driver;
use App\Models\Order;
use App\Models\Route;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;

class OrderWebController extends Controller
{
    public function index(): View
    {
        return view('orders.index', [
            'orders' => Order::with(['deliveryCenter', 'driver', 'route'])->latest()->paginate(10),
        ]);
    }

    public function create(): View
    {
        return view('orders.create', [
            'centers' => DeliveryCenter::where('status', 'active')->orderBy('name')->get(),
            'drivers' => Driver::where('status', '!=', 'offline')->orderBy('name')->get(),
            'routes' => Route::orderBy('route_name')->get(),
        ]);
    }

    public function store(Request $request): RedirectResponse
    {
        $data = $this->validatedData($request);
        $data['order_number'] = 'ORD-' . str_pad(Order::max('id') + 1, 3, '0', STR_PAD_LEFT);

        Order::create($data);

        return redirect()->route('orders.index')->with('success', 'Order created successfully.');
```

Figure 8: OrderWebController code for order management.

12.4 ORDERS MIGRATION

The orders migration defines the structure of the orders table, including customer information, order number, delivery address, status, and foreign keys for delivery center, driver, and route.

```
Orders Migration - database\migrations\2026_04_27_093449_create_orders_table.php

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('orders', function (Blueprint $table) {
            $table->id();
            $table->foreignId('delivery_center_id')->constrained()->cascadeOnDelete();
            $table->foreignId('route_id')->nullable()->constrained()->nullOnDelete();
            $table->foreignId('driver_id')->nullable()->constrained()->nullOnDelete();
            $table->string('order_number')->unique();
            $table->string('customer_name');
            $table->string('customer_phone');
            $table->string('customer_email')->nullable();
            $table->string('delivery_address');
            $table->string('items_summary')->nullable();
            $table->enum('status', ['pending', 'assigned', 'picked_up', 'delivered'])->default('pending');
            $table->timestamp('created_at')->nullable();
        });
    }

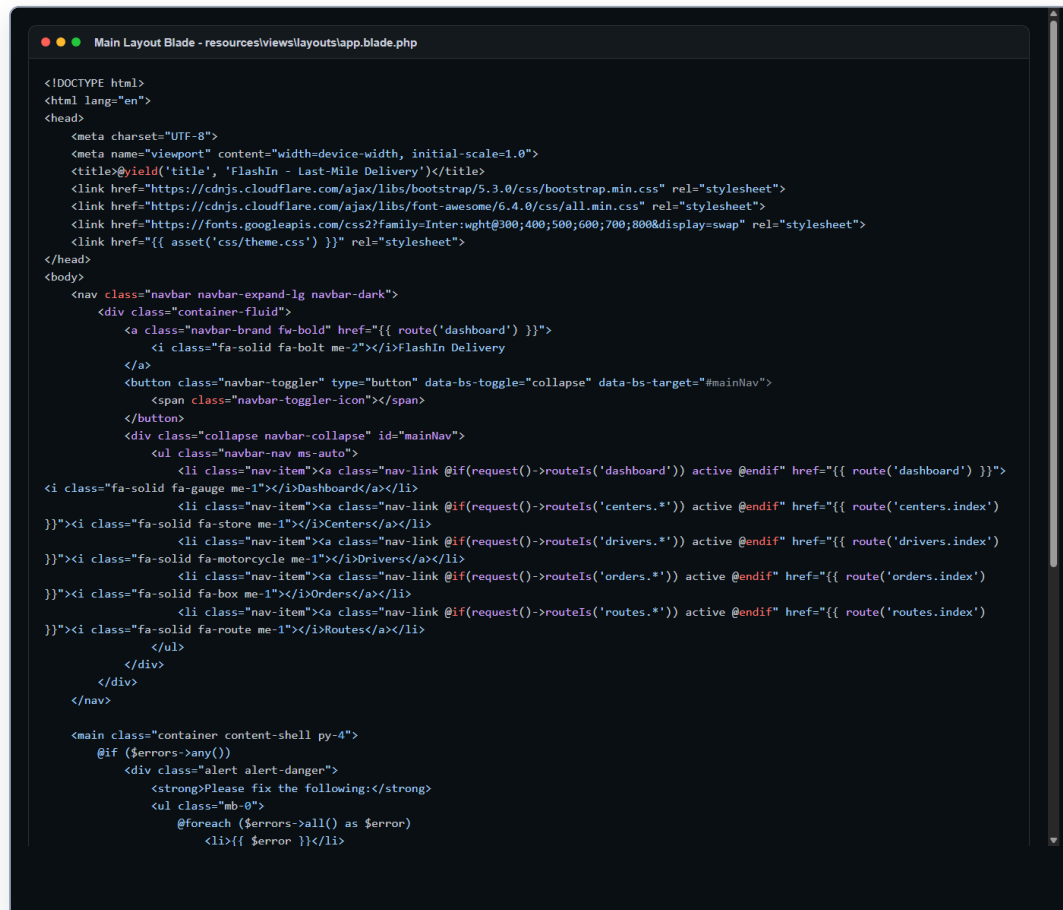
    public function down(): void
    {
        Schema::dropIfExists('orders');
    }
};
```

Figure 9: Orders table migration code.

12. CODE SCREENSHOTS CONTINUED

12.5 MAIN LAYOUT BLADE FILE

The layout file contains the common HTML structure used by all pages. It includes Bootstrap, Font Awesome, the custom theme file, navigation bar, alert messages, content section, and footer.



```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>@yield('title', 'FlashIn - Last-Mile Delivery')</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
    <link href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap" rel="stylesheet">
    <link href="{{ asset('css/theme.css') }}" rel="stylesheet">
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-dark">
        <div class="container-fluid">
            <a class="navbar-brand fw-bold" href="{{ route('dashboard') }}">
                <i class="fa-solid fa-bolt me-2"></i>FlashIn Delivery
            </a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#mainNav">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="mainNav">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item"><a class="nav-link @if(request()->routeIs('dashboard')) active @endif" href="{{ route('dashboard') }}">
                        <i class="fa-solid fa-gauge me-1"></i>Dashboard</a></li>
                    <li class="nav-item"><a class="nav-link @if(request()->routeIs('centers.*')) active @endif" href="{{ route('centers.index') }}">
                        <i class="fa-solid fa-store me-1"></i>Centers</a></li>
                    <li class="nav-item"><a class="nav-link @if(request()->routeIs('drivers.*')) active @endif" href="{{ route('drivers.index') }}">
                        <i class="fa-solid fa-motorcycle me-1"></i>Drivers</a></li>
                    <li class="nav-item"><a class="nav-link @if(request()->routeIs('orders.*')) active @endif" href="{{ route('orders.index') }}">
                        <i class="fa-solid fa-box me-1"></i>Orders</a></li>
                    <li class="nav-item"><a class="nav-link @if(request()->routeIs('routes.*')) active @endif" href="{{ route('routes.index') }}">
                        <i class="fa-solid fa-route me-1"></i>Routes</a></li>
                </ul>
            </div>
        </div>
    </nav>

    <main class="container content-shell py-4">
        @if ($errors->any())
            <div class="alert alert-danger">
                <strong>Please fix the following:</strong>
                <ul class="mb-0">
                    @foreach ($errors->all() as $error)
                        <li>{{ $error }}</li>
                    @endforeach
                </ul>
            </div>
        @endif
    </main>

```

Figure 10: Main Blade layout file.

12.6 PROFESSIONAL THEME CSS

The custom CSS file defines the professional theme, including the header, cards, metric boxes, buttons, forms, tables, badges, alerts, pagination, footer, and mobile responsiveness.

```
Professional Theme CSS - publiccssltheme.css

/* =====
FlashIn Delivery - Professional Operations Theme
===== */

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap');

:root {
  --page-bg: #f4f7fb;
  --surface: #ffffff;
  --surface-soft: #f8fafc;
  --ink: #172033;
  --ink-soft: #475569;
  --ink-muted: #738195;
  --line: #dce5ef;
  --line-strong: #c7d3e0;
  --nav: #0f2a37;
  --nav-soft: #17324a;
  --primary: #0f766e;
  --primary-dark: #115e59;
  --primary-soft: #e6f5f3;
  --blue: #2563eb;
  --blue-soft: #e8f0ff;
  --green: #15803d;
  --green-soft: #e9f7ef;
  --amber: #b45309;
  --amber-soft: #fff3dc;
  --red: #b91c1c;
  --red-soft: #fdecec;
  --slate-soft: #edf2f7;
  --shadow-sm: 0 1px 2px rgba(15, 35, 55, 0.06);
  --shadow-md: 0 12px 28px rgba(15, 35, 55, 0.09);
  --radius-sm: 6px;
  --radius-md: 8px;
}

* {
  box-sizing: border-box;
}

html,
body {
  min-height: 100%;
}
```

Figure 11: Custom professional theme stylesheet.

13. CONCLUSIONS AND FUTURE DIRECTIONS

13.1 SUMMARY OF FINDINGS

The FlashIn Delivery project successfully demonstrates a working last-mile delivery management system. It provides clear modules for centers, drivers, orders, and routes. The dashboard gives useful summary information, and the professional interface makes the website suitable for academic presentation.

The project proves that even a small web application can improve visibility in delivery operations. By using a centralized dashboard and CRUD modules, a business can avoid scattered records and track its operational flow more clearly. The project also shows the importance of structured database relationships because orders, routes, centers, and drivers depend on each other in real-world delivery management.

13.2 RECOMMENDATIONS FOR ENHANCEMENTS

- Add user authentication and role-based access for administrators and dispatchers.
- Add automated route optimization using distance matrix APIs.
- Add live map tracking for drivers and orders.
- Add customer-side order tracking with order status updates.
- Add delivery reports, charts, and exportable analytics.
- Add payment status and invoice generation.

13.3 FUTURE SCOPE OF THE WEBSITE

In the future, FlashIn can be expanded into a complete logistics platform. Mobile apps can be added for drivers and customers. Real-time location sharing, optimized multi-stop routing, notification systems, and advanced analytics can make the system more powerful and industry-ready.

14. REFERENCES

- Laravel Documentation: <https://laravel.com/docs>
- Bootstrap Documentation: <https://getbootstrap.com/docs>
- PHP Documentation: <https://www.php.net/docs.php>
- SQLite Documentation: <https://www.sqlite.org/docs.html>
- Font Awesome Icons: <https://fontawesome.com>
- GitHub Documentation: <https://docs.github.com>

15. GITHUB LINK

Repository URL:

<https://github.com/ShaikNazeer0073/Last-mile-route-planning.git>

16. PROJECT SETUP COMMANDS

Command	Purpose
composer install	Install PHP dependencies.
copy .env.example .env	Create environment configuration file.
php artisan key:generate	Generate Laravel application key.
php artisan migrate:fresh --seed	Create tables and insert demo data.
php artisan serve	Run the project locally.